

OLX Kerala Car Notifier

A self-hosted alert system that watches OLX for newly posted cars matching your filters and pings you on Telegram — before other buyers get there.

Status as of 31 Aug 2026 · 22/22 tests passing · verified against live OLX

How a check runs (every 5 min ± jitter)

Filters	Location	Fetch	Refine	Dedupe
Web UI saves to filters.json — reloaded each cycle	→ City name → OLX slug, cached after first resolve	→ One JSON API call from inside a real-Chrome page, 100 newest	→ Strict location by city ID + model / transmission / seller checks	→ seen_ads.json, pruned after 60 da

Module board

MODULE	STATE	NOTES
main.py	Working	5-min scheduler with ±90s jitter and exponential backoff after failed checks (skip 1 → 3 → 7 cycles).
webapp.py	Working	Filter manager on port 5001: location, price, year, km, brand, model, fuel, transmission, seller type, keyword. Changes apply next cycle, no restart.
scraper.py	Working	Headless real Chrome + normal user agent (passes OLX's block). Primary source is OLX's internal JSON API; HTML parsing is the automatic fallback. Session cookies persist between runs.
olx_api.py	Working	Builds the API query — verified params for make, price, year, mileage, fuel, keyword. Brand aliases ("maruthi" → maruti-suzuki).
olx_location.py	Working	Resolves city → slug by reading the link href (no fragile clicking); cached in location_cache.json.

<code>filters.py</code>	Working	Local safety net for what the API can't filter server-side: model, transmission, seller type — using structured fields, not title guessing.
Strict location	Working	OLX pads searches with nearby districts; listings are now matched by city ID (74 of 100 dropped in the live Kozhikode test).
<code>notifier.py</code> / <code>storage.py</code>	Working	Telegram HTML messages incl. seller line; seen-ads store prunes entries older than 60 days.
Rare-car filter	On hold	Started (rarity = model's total listing count), paused mid-build when the location bug took priority.
24/7 operation	Gap	Runs only while the terminal is open; Mac sleep pauses it. Headless mode makes a service/VPS possible — not yet set up.

Known limitations

- Model and transmission can't be filtered server-side by OLX's API — enforced locally instead (works, but rare models depend on being inside the newest 100 fetched).
- First run after changing filters sends a burst of notifications for the existing backlog.
- The HTML fallback path has no seller or location-ID data, so those checks pass everything through when the API is down.
- KYC-verified sellers are rare in the live data — the "verified" filter options may match almost nothing.
- The old Telegram token is still in git history; regenerate it via BotFather before the repo ever goes public.
- The web UI has no authentication and listens on all interfaces — fine at home, not on a public server.

Roadmap

TIER 1 · DO NEXT

Make it dependable

- **Run as a service** (launchd on the Mac mini, or a small VPS). Everything else is pointless if the bot is off when the right car is posted.

- **Health alerts** — a Telegram message when checks keep failing. Silent death is the main failure mode of any scraper.
- **Finish the rare-car filter** — the paused module. Already half-built and scoped.

TIER 2 · BIG VALUE ADDS

Better alerts, wider view

- **Photos in Telegram** — the API already returns image URLs; `sendPhoto` makes alerts instantly judgeable.
- **Multiple filter profiles** — watch a Swift 2015 and a rare-car search at the same time, each labeled in its alerts.
- **One-time full market sweep** — paginate everything matching a filter once, on demand, to see the whole market before alerts take over.
- **Nearby-areas toggle** — opt back into OLX's relaxed radius when strict location is too narrow.

TIER 3 · LATER

From notifier to market tool

- **SQLite + price history** — keep every sighting; flag re-posts and price drops on the same car.
- **Deal scoring** — compare a listing's price to the average for its model/year and mark underpriced ones.
- **Telegram bot commands** — change filters, pause, or ask for status from chat instead of the web UI.
- **Results dashboard** — a page in the web app showing matched cars with photos, instead of scrolling Telegram.